

Day 3 — ML, data, NLP, deep learning, and LLM foundations

Outcome

Be able to translate a business problem into a data and model problem, choose meaningful metrics, prevent leakage, explain transformer/LLM foundations, and decide between prompting, retrieval, tools, and fine-tuning.

1. From business problem to ML problem

Start with the decision or outcome, not an algorithm.

Business goal:
Reduce customer churn.

ML formulation:
Binary classification predicting whether a customer will leave.

Business action:
Retention team contacts high-risk customers.

Learning paradigms

Paradigm	Input	Goal	Examples
Supervised	Inputs plus labels	Learn $X \rightarrow y$	Churn, fraud, price, intent, toxicity.
Unsupervised	Inputs without labels	Discover structure	Clustering tickets/documents, anomaly detection.
Reinforcement learning	State, actions, rewards	Learn a policy	Preference alignment concept; not the same as an ordinary tool-using agent.

Classification predicts a category; regression predicts a continuous number; clustering groups similar examples.

Traditional ML usually learns one narrower task from task-specific features and labels. A foundation model is pretrained broadly, then reused through prompting, retrieval, tools, or adaptation. The broader model is flexible but also larger, less deterministic, more expensive, and still needs task-specific evaluation and controls.

2. Data splits, leakage, and generalization

- Training data learns parameters.
- Validation data selects models, thresholds, and hyperparameters.
- Test data estimates final generalization.
- Cross-validation repeats training/evaluation across folds when appropriate.

Use time-aware splits for future prediction and group-aware splits when documents, users, or conversations could leak across sets.

In K-fold cross-validation, each fold becomes validation data once while the remaining folds train the model. It produces a more stable estimate than relying on one lucky split, especially for smaller datasets. Learned preprocessing must be fitted inside each training fold; fitting it once on all data leaks validation information.

Leakage test

Ask:

| Would this information exist at the moment the prediction is made?

Bad churn feature: `account_closure_date`.

Bad loan-default feature: late payments recorded after loan approval.

GenAI leakage:

- chunks from one document in both train and test;
- golden questions inserted into prompt examples;
- evaluation examples included in fine-tuning data.

Underfitting and overfitting

- Underfitting/high bias: model is too simple for the pattern.
- Overfitting/high variance: excellent training performance, weak unseen performance.

Controls include more representative data, regularization, smaller capacity, limiting tree depth, dropout, early stopping, frozen parameters, and careful validation.

Bias, variance, and regularization

- High bias: the model is too simple and underfits.
- High variance: the model is too sensitive to its training sample and overfits.
- Reduce high bias with justified capacity or better features.
- Reduce high variance with representative data, L1/L2 or weight-decay penalties, dropout, early stopping, data augmentation, ensembling, pruning/depth limits, label smoothing, or freezing most pretrained parameters.

Choose a control that matches the model and observed failure; adding regularization to an already underfit model can make it worse.

3. Evaluation metrics

Classification

The confusion matrix separates:

- true positive: predicted positive and actually positive;
- false positive: predicted positive but actually negative;
- false negative: predicted negative but actually positive;
- true negative: predicted negative and actually negative.

Accuracy	= (TP + TN) / all examples
Precision	= TP / (TP + FP)
Recall	= TP / (TP + FN)
F1	= 2 × precision × recall / (precision + recall)

- Precision matters when false alarms are expensive.
- Recall matters when missing a positive is dangerous.
- F1 balances the two.
- Accuracy can be misleading for imbalanced data.
- ROC-AUC measures ranking quality across thresholds.

Metric selection follows business cost. A prompt-injection detector needs high recall for attacks and sufficient precision not to block normal work.

Regression

- MAE: average absolute error in the original unit.
- MSE: squares errors and penalizes large mistakes.
- RMSE: square root of MSE, again in the original unit.
- R²: variation explained by the model.

Production metrics

Offline model quality is not enough. Also track latency, errors, drift, cost, fairness, user feedback, and business impact.

Classical baselines

Linear regression predicts a continuous value as a weighted combination of features. It is interpretable and useful for cost, latency, and capacity baselines, but does not naturally model complex nonlinear relationships.

Logistic regression is primarily a classifier despite its name:

$p = 1 / (1 + e^{-z})$

It is useful for binary risk, intent, and interpretable probability baselines.

Decision trees split data with feature conditions. They handle nonlinear relationships and require little preprocessing, but overfit and can be unstable; limit depth and minimum samples.

Random forests average or vote across many sampled trees. They are stable tabular baselines that capture interactions, but are larger and less interpretable than one tree. Start with a baseline before choosing a more complex model.

4. Exploratory data analysis (EDA) and leakage-safe preprocessing

EDA checklist

- Shape, columns, types, target distribution.
- Missing values and why they are missing.
- Duplicate rows/entities.
- Invalid ranges and outliers.

- Cardinality and unseen categories.
- Leakage columns and identifiers.
- Relationships between features and target.

Classify variables before choosing transformations:

Variable type	Example	Consequence
Nominal categorical	City or department	No natural order; one-hot encoding is a common fit.
Ordinal categorical	Low, medium, high	Preserve the real order explicitly.
Discrete numeric	Number of children or tickets	Countable values.
Continuous numeric	Salary or temperature	Can take fractional values; inspect distribution and scale.

Missing values

- Drop rows only when loss is acceptable.
- Mean is sensitive to outliers.
- Median is safer for skewed numeric data.
- Mode or an explicit Unknown may fit categorical data.
- Missingness itself can carry meaning and should be understood.

Outliers

Outliers may be data errors or important rare events. Detect them with business rules, visual inspection such as a boxplot, or the IQR rule:

$$\begin{aligned} \text{IQR} &= Q3 - Q1 \\ \text{lower bound} &= Q1 - 1.5 \times \text{IQR} \\ \text{upper bound} &= Q3 + 1.5 \times \text{IQR} \end{aligned}$$

Then decide whether to fix/remove an invalid value, cap or transform a valid extreme value, or keep a meaningful rare case. Removing them blindly can delete the fraud signal.

Encoding

- One-hot: nominal categories without order.
- Ordinal encoding: categories with real order.
- Label encoding can create a false numeric order for nominal categories.
- Production encoders must handle unseen values.

Scaling

- Standardization centers/scales by training statistics.
- Normalization maps to a defined range.
- Fit imputer, encoder, and scaler on training data only; apply the fitted pipeline to validation/test/production.

Distance- and gradient-based methods such as KNN, SVM, linear/logistic regression, and neural networks usually benefit from scaling. Tree-based methods such as decision

trees, random forests, and XGBoost-style models usually do not require it because their splits use thresholds.

Feature engineering

Feature engineering converts raw fields into more useful signals:

Raw data	Engineered feature
Join and last-login dates	Tenure and days since last login.
Purchase history	Average order value or orders per month.
Transaction timestamp	Day, month, hour, day-of-week, weekend flag.
Loan and salary	Loan-to-income ratio.
Review text	Length, keywords, sentiment, or embeddings.

An identifier such as customer ID is usually not a meaningful behavior feature. Create features using only information available at prediction time, fit learned transformations on training data, and package the feature logic with the model.

Feature-like signals also appear in GenAI operations: chunk size, similarity/reranker scores, prompt length, latency, and feedback can support retrieval, monitoring, classification, or routing.

Reproducible flow

<pre>basic non-learning cleanup → split → fit preprocessing on train → transform train/validation/test → train → evaluate → package model plus preprocessing</pre>
--

The same preprocessing logic must run in training and production.

5. Python AI framework map

Framework	Center of gravity
NumPy	Arrays and numerical computation.
Pandas	Tabular loading, EDA, cleaning, and transformation.
scikit-learn	Classical ML, preprocessing pipelines, baselines, and metrics.
PyTorch	Deep-learning tensors, training, research, and custom models.
TensorFlow/Keras	Deep-learning models and higher-level training APIs.
Hugging Face	Pretrained transformer models, tokenizers, datasets, and pipelines.

Do not start with deep learning for every problem. Build a simple baseline, understand the data, and choose complexity based on the task and constraints.

Representative source-note choices:

Problem	Starting tools/models
Tabular classification	scikit-learn with logistic regression, decision tree, random forest, or an XGBoost-style model.
Tabular regression	Linear regression or a random-forest regressor.
Clustering	KMeans or DBSCAN.
Dimensionality reduction	PCA fitted only on training data.
Custom image, text, audio, or sequence model	PyTorch or TensorFlow/Keras.
Pretrained NLP/LLM task, embeddings, or fine-tuning	Hugging Face transformers, tokenizers, datasets, and PEFT components.

Framework selection example:

```
structured churn data
→ Pandas EDA
→ scikit-learn preprocessing pipeline
→ logistic-regression baseline
→ compare a random forest
→ record evaluation and the fitted preprocessing with the model
```

A tensor is the deep-learning analogue of a numerical array and can execute on accelerators. PyTorch emphasizes flexible custom training; Keras provides a higher-level TensorFlow model API. Organizational skills and deployment constraints matter alongside syntax.

6. NLP foundations

Structured data has a fixed schema such as rows and columns. Unstructured data includes free-form documents, emails, tickets, images, and audio; semi-structured formats preserve some organization without a fixed relational shape.

Traditional pipeline

```
raw text
→ cleaning
→ tokenization
→ optional stop-word/stemming/lemmatization
→ representation
→ task model
```

- Stop-word removal can delete important negation or policy terms.
- Stemming cuts words mechanically; lemmatization maps to a language-aware base form.
- Bag of Words counts terms but does not represent meaning or order.

- TF-IDF downweights terms common across documents.
- Word embeddings represent words; sentence embeddings represent larger meaning units.
- Modern LLM workflows often need less aggressive preprocessing.

Tasks:

- text classification;
- named-entity recognition;
- sentiment analysis;
- document grouping;
- semantic retrieval.

RAG connection:

```
document text
→ chunks
→ sentence/chunk embeddings
→ similarity retrieval
→ LLM answer
```

Preserve source and security metadata while processing text.

7. Math and neural-network refresh

- Vector: ordered numeric representation.
- Matrix: two-dimensional numeric collection.
- Dot product: alignment influenced by magnitude.
- Cosine similarity: directional similarity:

$$\cos(a, b) = (a \cdot b) / (||a|| \ ||b||)$$

- Gradient: how loss changes with parameters.
- Backpropagation: efficient chain-rule computation of gradients.
- Random variable: a numeric outcome governed by uncertainty.
- Expectation: probability-weighted average.
- Variance: spread around expectation.
- Conditional probability: probability of an event given known evidence.
- Bayes' rule: update a prior belief using evidence.

A layer applies:

$$\text{output} = \text{activation}(W \times \text{input} + b)$$

Activations such as ReLU, GELU, sigmoid, tanh, and softmax add nonlinearity. Without nonlinear activation, stacked linear layers still collapse to a linear transformation.

Loss measures the task error:

- cross-entropy for classification or next-token prediction;
- MSE for regression;
- contrastive loss for embeddings;
- preference loss for alignment.

Optimization updates parameters to reduce that loss.

SGD versus Adam

- SGD is simple and memory-light but sensitive to learning rate.
- Adam adapts updates per parameter and often converges faster initially, at additional optimizer state and memory.

8. Transformers

A transformer block combines attention, a feed-forward network, residual connections, and normalization.

Self-attention

Each token builds a contextual representation using other relevant tokens.

```
scores = QKT / √d
weights = softmax(scores)
output = weights × V
```

- Query: what information is this token seeking?
- Key: what information does another token advertise?
- Value: what content is returned when relevant?

Multi-head attention learns several interaction patterns. Positional information supplies order.

Architecture families

Family	Typical strengths
Encoder-only	Classification, NER, embeddings, matching.
Decoder-only	Chat, generation, code, tool selection.
Encoder-decoder	Translation, summarization, sequence-to-sequence work.

Transformer limits include compute cost, long-sequence expense, hallucination, bias, and imperfect use of large context.

9. LLM foundations

Training/adaptation stages

- Pretraining: broad language, knowledge, and pattern learning.
- Supervised fine-tuning: curated input-output behavior.
- Instruction tuning: following diverse instructions.
- RLHF: learn preferences from human comparisons and optimize toward them.
- DPO: train directly from preferred/rejected pairs using a preference objective.

Tokens and context window

LLMs operate on tokens, which may be words, subwords, punctuation, whitespace-text units, or byte pieces.

- BPE repeatedly builds a subword vocabulary from frequent symbol pairs.

- SentencePiece learns tokenization directly from raw text and can represent spaces as part of the token stream.

Subword tokenization balances reusable pieces with vocabulary size and supports unfamiliar words better than a word-only vocabulary.

Context includes:

```
system instructions
+ chat history
+ retrieved evidence
+ examples
+ tool descriptions/results
+ user input
+ generated output
```

Use the smallest sufficient, highest-quality context. A larger limit does not guarantee equal attention, lower latency, lower cost, or better reasoning.

Inference controls

- Temperature: randomness.
- Top-k: sample among the highest-probability k tokens.
- Top-p: sample from the smallest probability-mass set reaching p.
- Max tokens: output cap.
- Stop sequences: terminate on configured patterns.
- Repetition penalty: discourage repetition, but excessive penalty harms coherence.

Low variation is appropriate for extraction, tools, compliance, and structured output; more variation fits creative generation. Sampling settings cannot add missing knowledge.

Open-weight versus commercial/provider models

Open-weight/private deployment can offer control, locality, customization, and vendor flexibility, but requires infrastructure, security, monitoring, and lifecycle skills.

Managed APIs offer fast adoption, capability, and scaling, but introduce ongoing cost, rate limits, network dependency, governance review, and vendor behavior changes.

Compare total constraints: task quality, context, tools, structure, multilingual/multimodal support, latency, throughput, license, hosting, cost, and data terms.

10. Prompt engineering and guardrails

Message roles

- System message: role, boundaries, tools, output contract, abstention, and safety behavior.
- User message: user request and potentially untrusted data.
- Assistant message: prior responses or approved examples.
- Tool message: result from an external operation.

System instructions influence behavior but are not authorization. Application and tool services enforce permissions.

Production prompts

Define:

- task and authoritative sources;
- untrusted-data boundaries;
- allowed tools and approval rules;
- output schema;
- citations and abstention;
- safety constraints.

Move deterministic policy into code. Avoid huge prompts with conflicting or outdated rules.

Few-shot, chain-of-thought, and ReAct

Few-shot examples help classification, extraction, format, tool choice, and edge cases, but consume tokens and can teach bad behavior.

For complex reasoning, ask for concise evidence, assumptions, verification steps, or a structured rationale rather than depending on exposure of private internal chain-of-thought. Prefer calculators, code, solvers, and validators for deterministic work.

ReAct-style execution alternates conceptually:

```
understand
→ choose action
→ call tool
→ observe
→ choose next step
→ answer
```

Keep production control state structured; do not parse free-form Thought: strings. Validate tools, bound iterations, and record traces.

Structured output and prompt guardrails

Use JSON Schema, typed objects, function/tool calling, grammar constraints, or post-generation validation. Validate required fields, types, enum values, lengths, business invariants, evidence, and authorization.

Prompt rules such as “use only evidence,” “do not follow document instructions,” and “require approval” help behavior but need access control, tool allowlists, validation, sanitization, sandboxing, monitoring, and human approval.

Prompt regression testing

Cover:

- happy and edge cases;
- injections and unsafe inputs;
- empty/conflicting retrieval;
- multilingual/long inputs;
- malformed tool output;
- citation, refusal, schema, trajectory, latency, token, and cost behavior.

11. Prompting, RAG, tools, or fine-tuning?

Need	Primary mechanism
Temporary role, rules, or format	Prompting
Current/private/large factual knowledge	RAG
Live transactional state or actions	Tools/APIs
Stable repeated behavior/style/task pattern	Fine-tuning

Decision sequence:

Can prompting solve it?
→ Can retrieval or a tool solve it?
→ Can constrained output solve it?
→ Is the desired behavior stable and repeated?
→ Is high-quality training data available?
→ Consider fine-tuning.

Fine-tuning does not repair broken parsing, missing authorization, poor retrieval, incorrect tool logic, or frequently changing facts.

PEFT, LoRA, and QLoRA

- PEFT updates a small subset or added adapter parameters.
- LoRA learns low-rank updates while base weights remain frozen.
- QLoRA keeps the base model quantized while training LoRA adapters.

Benefits are lower memory and smaller artifacts; the work still depends on representative clean data and evaluation.

Fine-tuning risks:

- overfitting;
- catastrophic forgetting;
- noisy or contradictory data;
- test contamination;
- incorrect root-cause diagnosis.

Adaptation categories:

- Domain adaptation: terminology and domain patterns.
- Style tuning: tone and formatting.
- Task-specific tuning: classification, extraction, SQL, routing, or tool invocation.
- Preference alignment: preferred/rejected pairs shaping subjective quality or policy behavior.

Training data should be representative, deduplicated, scrubbed of unapproved PII/secrets, balanced, include hard negatives and abstentions, split without leakage, and versioned.

12. Multimodal and generative models

- CNNs learn local spatial patterns; transfer learning adapts a pretrained vision model to a smaller proprietary task.

- Multimodal LLMs combine inputs such as text, images, or documents; architecture must still decide which modality is authoritative.
- Diffusion generation learns to reverse a noising process.
- VAE, GAN, diffusion, and LLM families solve different generative problems and have different training/inference behavior.

Risks include hallucination, bias, toxicity, copyright/provenance, and weak evaluation. Use human review and task-specific evaluation where risk is high.

Generative evaluation

- BLEU measures n-gram overlap, traditionally for translation; correct paraphrases may score poorly.
- ROUGE measures overlap, often recall-oriented for summarization; overlap does not prove factuality.
- LLM-as-judge scales rubric-based scoring but can show position, verbosity, self-preference, prompt, correlated-error, and judge-drift biases.
- Human evaluation is strongest for high-risk, nuanced, brand, safety, and user-experience judgments.

Combine deterministic metrics, model-based judging, and human review rather than using one signal alone.

13. Production ML lifecycle

```
business understanding
→ data collection/EDA
→ preprocessing/features
→ split and baseline
→ train/tune/evaluate
→ package/version
→ deploy
→ monitor data, model, system, and business
→ retrain or roll back
```

- Data drift: the production input distribution changes.
- Concept drift: the relationship between input and target/outcome changes.
- Model-performance degradation: observed quality declines. It may be caused by data drift, concept drift, pipeline defects, label changes, or an unsuitable threshold.
- Version model, data, preprocessing, code, dependencies, and evaluation.

The phrase “model drift” is ambiguous across teams. In an interview or monitoring contract, define whether it means concept drift or measured model-performance degradation.

Project-grounded examples

Scenario 1: recognizing when deterministic data engineering is the right solution

Project scenario. The first **DPDK Automation for Network Packet Processing** project had a concrete business decision: compare AMD CPU performance across workloads, CPU SKUs, BIOS settings, operating systems, and compilers. Raw DPDK output and CPU statistics were converted by workload-specific parsers and a Python

processing module into structured metrics for database-backed graphs and run comparisons.

How the concepts apply. The correct formulation was initially automation and data normalization, not model training. The command formats and performance fields were sufficiently defined to use deterministic parsers, while the outcome required trustworthy comparisons rather than a learned prediction. This is the “start with the business decision, not an algorithm” principle in practice.

Decision and trade-offs. Custom parsers required maintenance when benchmark output changed, but they provided explicit, reproducible metric extraction for testpmd, crypto, and vhost. A single generic parser would have been simpler to describe but less able to preserve workload-specific semantics. The structured data then became a stronger substrate for later analysis and AI assistance.

Senior/Staff interview framing.

- **Senior:** explain the input schema, normalization rules, invalid-output behavior, and how you verified that two runs were comparable.
- **Staff:** explain why you deliberately did not introduce ML for a deterministic problem, and how investing in clean operational data created the option to add higher-level intelligence later.

Scenario 2: choosing RAG and tools instead of fine-tuning

Project scenario. In **DPDK BenchOps Copilot**, engineers needed current/private benchmark guidance, historical run context, exact commands, and safe comparisons. The architecture used RAG for factual benchmark/tuning knowledge, an LLM for question answering and synthesis, and deterministic tools for run/log access, comparison, validation, parsing, and command construction. The project narrative does not identify a fine-tuned model.

How the concepts apply. This maps directly to the mechanism-selection table:

private and changing guides/logs/run context	→ RAG
live run data and comparisons	→ tools
safe repeatable command construction	→ deterministic templates/tools
natural-language explanation	→ LLM prompting and synthesis

Fine-tuning would not have solved knowledge freshness, citations, authorization, or safe command execution.

Decision and trade-offs. RAG and tools introduced retrieval, metadata, orchestration, evaluation, and dependency latency, but preserved source provenance and let authoritative data change without retraining. The design also constrained the model: it could synthesize evidence but could not become the source of operational truth.

Outcome. The documented outcome was grounded assistance with citations, faster analysis, reduced reliance on tribal knowledge, and safer workflows. No quantitative latency, accuracy, or productivity improvement is recorded, so do not invent one.

Senior/Staff interview framing.

- **Senior:** walk through one regression question and show which facts came from retrieval, which values came from RunDiff, and what the LLM was allowed to summarize.

- **Staff:** frame the choice as failure-mode allocation: freshness and provenance went to RAG, live truth and side effects to tools, and language synthesis to the model. Then explain the evaluation metrics—groundedness, context precision/recall, citation coverage, tool success/error rate, and p95 latency—that governed releases.

14. Interview questions

1. Supervised versus unsupervised learning?
2. Why separate validation and test sets?
3. Why is accuracy weak for an imbalanced attack detector?
4. How do precision and recall express business cost?
5. What is leakage, including in RAG/fine-tuning data?
6. How do you make preprocessing identical in training and production?
7. Dot product versus cosine similarity?
8. What does backpropagation compute?
9. Explain self-attention and query/key/value.
- 10 . Encoder-only versus decoder-only versus encoder-decoder?
- 11 . Why use subword tokenization?
- 12 . Why does long context not replace RAG?
- 13 . Pretraining versus instruction tuning versus fine-tuning?
- 14 . When is LoRA/QLoRA appropriate?
- 15 . Provider API versus self-hosting?
- 16 . What makes linear/logistic regression, trees, and forests useful baselines?
- 17 . What belongs in system, user, assistant, and tool messages?
- 18 . Why are prompt guardrails not a security boundary?
- 19 . BLEU/ROUGE versus LLM-as-judge versus human evaluation?
- 20 . How can preprocessing leak information during cross-validation?
- 21 . How do high bias and high variance lead to different interventions?

15. Exit checklist

- [] Formulate classification, regression, clustering, and RL concepts.
- [] Select metrics from error cost.
- [] Detect data and evaluation leakage.
- [] Build a leakage-safe preprocessing pipeline.
- [] Explain the Python AI framework ecosystem.
- [] Explain NLP representations and tasks.

- [] Explain gradients, attention, transformer families, tokens, and context.
- [] Choose prompting, RAG, tools, or fine-tuning by failure cause.
- [] Explain deployment, drift, retraining, and rollback.
- [] Explain prompt roles, structured output, guardrails, and regression tests.
- [] Compare generative evaluation methods and their failure modes.
- [] Distinguish data drift, concept drift, and measured performance degradation.

Source notes

- [ML Fundamentals](#)
- [Python AI Frameworks](#)
- [EDA and Preprocessing](#)
- [NLP Fundamentals](#)
- [LLM Fundamentals](#)
- [Capstone Revision Day 2](#)
- [DPDK Automation for Network Packet Processing](#)
- [DPDK BenchOps Copilot](#)